

Project 1

Bibek Adhikari, Sai Ram Vodnala

2025-02-26

Project Description

We are interested in incorporating typing dynamics into our password based security system. See the included paper for additional background on this type of biometric research. In prior work (namely last semester) it was noted that a user's typing dynamics change over time when they type a given passcode. We need to incorporate this information into the biometric system by finding a lower threshold (in terms of the number of times a person has typed a passcode in the past) with which we can start training the system.

For this project you are given access to a group of 51 individuals who have typed a passcode for a specific system.

The passcode used by this code is : *.tie5Roanl*

Dr. Roy Maxian and colleagues recruited 51 subjects at CMU. Subjects completed 8 data collection sessions, with each session composed of 50 times of typing the above passcode. Each session was one day apart. There is additional details concerning the data collection in the included paper.

FOR EACH SUBJECT, YOUR TASK IS TO FIND A SUBSET OF THEIR TYPING DATA THAT IS REPRESENTATIVE OF THE LEARNED TYPING BEHAVIOR.

This problem is going to be broken into the 4 following data analysis components or tasks.

[Sub-Task 1:] Develop a covariate that tells you the number of times a subject has typed the passcode in the past.

[Sub-Task 2:] For each subject, build three appropriate regression models for the total UD, total DD, and total H times in terms of the number of times the passcode has been typed.

[Sub-Task 3:] Use FDR's to assess reasonableness of the normality assumptions of the residuals after fitting model for all subjects.

[Sub-Task 4:] Draw conclusions and make recommendations based on your analysis.

The results of your analysis should be presented in a short white paper and 15 to 30 minute powerpoint slide deck and presentation scheduled during the first week of February.

Task-1: Covariate development

```
# Loading the passcode dataset
Password_data <- read.csv("DSL-StrongPasswordData.csv", header = TRUE)
head(Password_data)
```

```
##  subject sessionIndex rep H.period DD.period.t UD.period.t  H.t DD.t.i
## 1    s002             1   1   0.1491      0.3979      0.2488 0.1069 0.1674
## 2    s002             1   2   0.1111      0.3451      0.2340 0.0694 0.1283
## 3    s002             1   3   0.1328      0.2072      0.0744 0.0731 0.1291
```

```
## 4      s002          1  4  0.1291      0.2515      0.1224 0.1059 0.2495
## 5      s002          1  5  0.1249      0.2317      0.1068 0.0895 0.1676
## 6      s002          1  6  0.1394      0.2343      0.0949 0.0813 0.1299
##      UD.t.i      H.i DD.i.e UD.i.e      H.e DD.e.five UD.e.five H.five DD.five.Shift.r
## 1 0.0605 0.1169 0.2212 0.1043 0.1417      1.1885      1.0468 0.1146      1.6055
## 2 0.0589 0.0908 0.1357 0.0449 0.0829      1.1970      1.1141 0.0689      0.7822
## 3 0.0560 0.0821 0.1542 0.0721 0.0808      1.0408      0.9600 0.0892      0.6203
## 4 0.1436 0.1040 0.2038 0.0998 0.0900      1.0556      0.9656 0.0913      1.2564
## 5 0.0781 0.0903 0.1589 0.0686 0.0805      0.8629      0.7824 0.0742      0.8955
## 6 0.0486 0.0744 0.1412 0.0668 0.0863      0.9373      0.8510 0.0942      1.0896
##      UD.five.Shift.r H.Shift.r DD.Shift.r.o UD.Shift.r.o      H.o DD.o.a UD.o.a
## 1      1.4909      0.1067      0.7590      0.6523 0.1016 0.2136 0.1120
## 2      0.7133      0.1570      0.7877      0.6307 0.1066 0.1684 0.0618
## 3      0.5311      0.1454      0.7195      0.5741 0.1365 0.2931 0.1566
## 4      1.1651      0.1454      0.7550      0.6096 0.0956 0.1530 0.0574
## 5      0.8213      0.1243      0.7632      0.6389 0.0430 0.1975 0.1545
## 6      0.9954      0.1681      0.3716      0.2035 0.1154 0.1287 0.0133
##      H.a DD.a.n UD.a.n      H.n DD.n.l UD.n.l      H.l DD.l.Return UD.l.Return
## 1 0.1349 0.1484 0.0135 0.0932 0.3515 0.2583 0.1338      0.3509      0.2171
## 2 0.1412 0.2558 0.1146 0.1146 0.2642 0.1496 0.0839      0.2756      0.1917
## 3 0.1621 0.2332 0.0711 0.1172 0.2705 0.1533 0.1085      0.2847      0.1762
## 4 0.1457 0.1629 0.0172 0.0866 0.2341 0.1475 0.0845      0.3232      0.2387
## 5 0.1312 0.1582 0.0270 0.0884 0.2517 0.1633 0.0903      0.2517      0.1614
## 6 0.1272 0.1534 0.0262 0.0858 0.2528 0.1670 0.0792      0.2971      0.2179
##      H.Return
## 1      0.0742
## 2      0.0747
## 3      0.0945
## 4      0.0813
## 5      0.0818
## 6      0.0591
```

#Creating a variable that counts total number of times a passcode has been typed by a subject and calcu

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##      filter, lag

## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union

Total_attempts <- Password_data |>
  group_by(subject) |> # Group data by subject
  mutate(typing_count = row_number(), # Create a new variable counting typing attempts
         total_h_time = rowSums(across(starts_with("H."))), # Create total hold time
         total_ud_time = rowSums(across(starts_with("UD."))), # Create total up-down time
         total_dd_time = rowSums(across(starts_with("DD."))) |> # Create total down-down time
  ungroup() # Ungroup
```

```

# Create a single graph with individual boxplots for specified subjects for EDA
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v forcats 1.0.0      v readr 2.1.5
## v ggplot2 3.5.1      v stringr 1.5.1
## v lubridate 1.9.4    v tibble 3.2.1
## v purrr 1.0.4       v tidyr 1.3.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

library(lme4)

## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

library(ggpubr)
boxplot_plots <- ggarrange(
  ggplot(Total_attempts %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_ud_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
  labs(x = "sessionIndex", y = "Total UD Time", title = "Total UD Time vs Subject") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 10, face = "bold")),

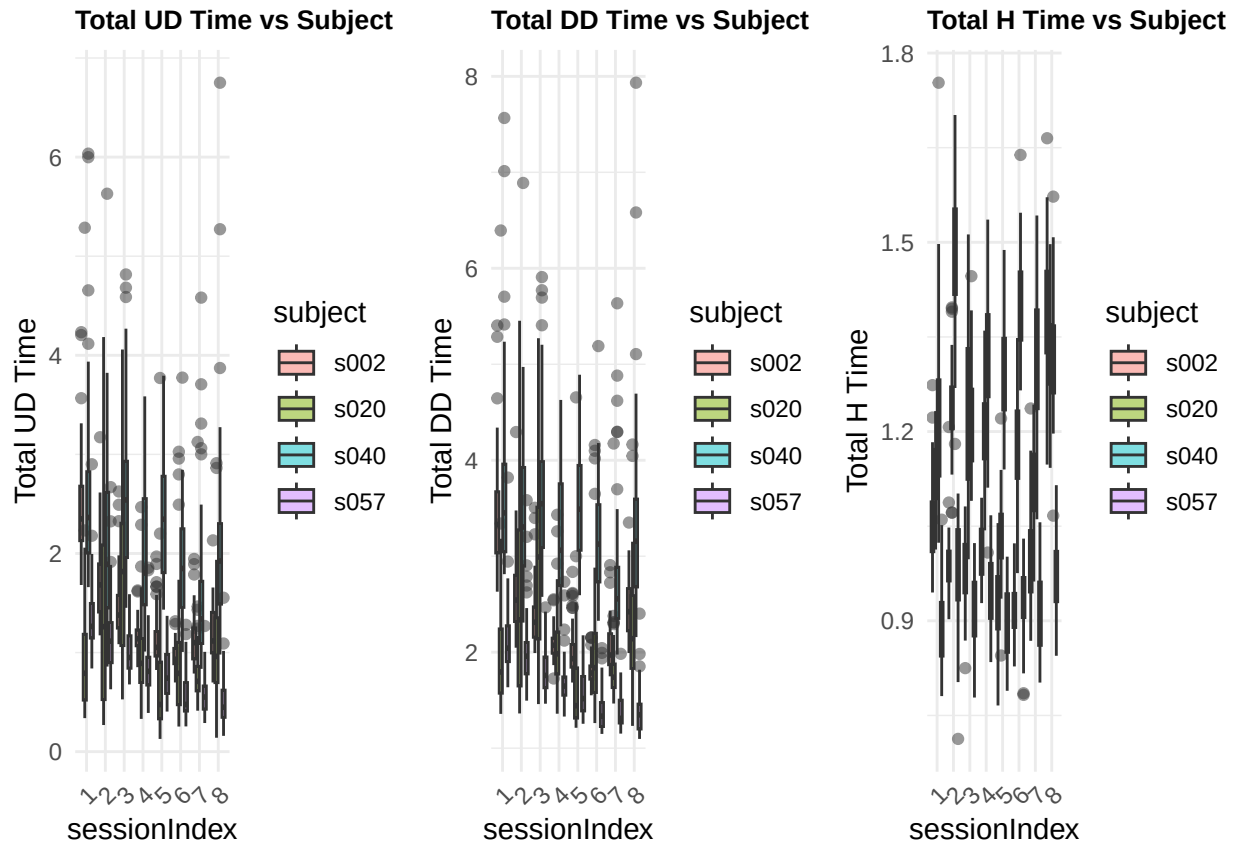
  ggplot(Total_attempts %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_dd_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
  labs(x = "sessionIndex", y = "Total DD Time", title = "Total DD Time vs Subject") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 10, face = "bold")),

  ggplot(Total_attempts %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_h_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
  labs(x = "sessionIndex", y = "Total H Time", title = "Total H Time vs Subject") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 10, face = "bold")),

  ncol = 3 # Arrange in 3 columns
)

# Display the plot
boxplot_plots

```



```
# Filtering the dataset to get only the top 20 passcode entries per session
```

```
library(tidyr)
model_data <- Total_attempts |>
  filter(rep >= 31) # Filtering the dataset to get top 20 passcode entries in each session
```

```
# EDA for filtered subjects
```

```
boxplot_plots <- ggarrange(
  ggplot(model_data %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_ud_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
  labs(x = "sessionIndex", y = "Total UD Time", title = "Total UD Time vs Subject") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 10, face = "bold")),

  ggplot(model_data %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_dd_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
  labs(x = "sessionIndex", y = "Total DD Time", title = "Total DD Time vs Subject") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
    plot.title = element_text(size = 10, face = "bold")),

  ggplot(model_data %>% filter(subject %in% c("s002", "s020", "s040", "s057")),
    aes(x = as.factor(sessionIndex), y = total_h_time, fill = subject)) +
  geom_boxplot(alpha = 0.5) +
```

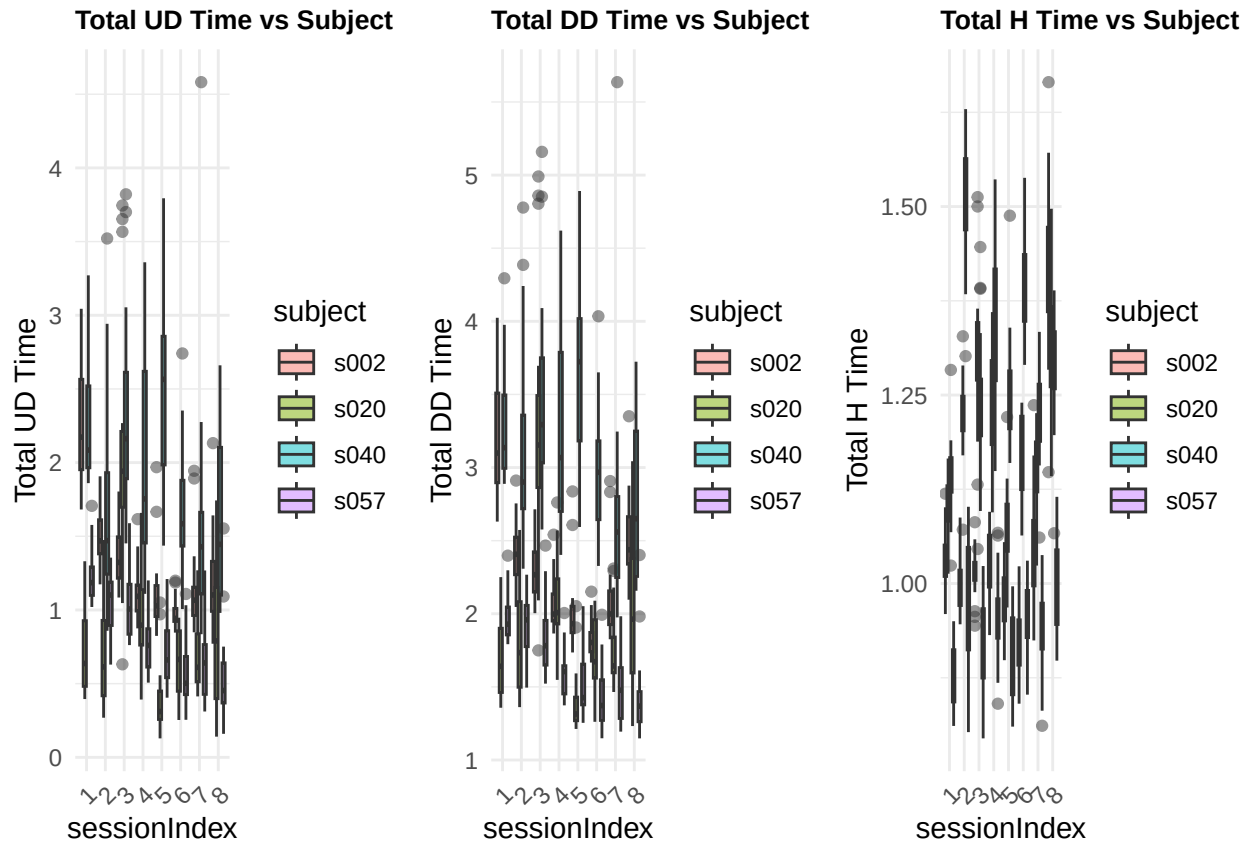
```

labs(x = "sessionIndex", y = "Total H Time", title = "Total H Time vs Subject") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1),
      plot.title = element_text(size = 10, face = "bold")),

ncol = 3 # Arrange in 3 columns
)

# Display the plot
boxplot_plots

```



Task-2: Building Regression Model

##To build regression model we filtered the data and considered only the top twenty password typings of each subject for each session

Regression models for each subject to create variable that can be view as the amount of time needed t
We used linear models

```

models <- model_data |>
  nest() |> # Nesting the data
  mutate(
    UD_model = map( # Fitting the linear model for up-down
      data, # Subject are second level variable
      ~ lm(total_ud_time ~ typing_count + sessionIndex, data = .x)),
    DD_model = map( # Fitting the linear model for down-down time
      data, # Subject are second level variable

```

```

    ~ lm(total_dd_time ~ typing_count + sessionIndex, data = .x)),
  H_model = map( # Fitting the linear model for Holding
    data, # Subject are second level variable
    ~ lm(total_h_time ~ typing_count + sessionIndex, data = .x))) |>
mutate(
  UD_model_summary = map(UD_model, summary), # Get summary of UD_model
  DD_model_summary = map(DD_model, summary), # Get summary of DD_model
  H_model_summary = map(H_model, summary)    # Get summary of H_model
)

# Print the summaries
cat("Summary for UD model:\n")

```

```
## Summary for UD model:
```

```
models$UD_model_summary
```

```

## [[1]]
##
## Call:
## lm(formula = total_ud_time ~ typing_count + sessionIndex, data = .x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.5865 -0.6635 -0.2377  0.3305 12.6507
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.018001   0.031549  63.964 <2e-16 ***
## typing_count    0.001870   0.001988   0.940  0.3470
## sessionIndex -0.205215   0.099545  -2.062  0.0393 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.036 on 8157 degrees of freedom
## Multiple R-squared:  0.05768,    Adjusted R-squared:  0.05745
## F-statistic: 249.7 on 2 and 8157 DF,  p-value: < 2.2e-16
cat("Summary for DD model:\n")

```

```
## Summary for DD model:
```

```
models$DD_model_summary
```

```

## [[1]]
##
## Call:
## lm(formula = total_dd_time ~ typing_count + sessionIndex, data = .x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.6191 -0.6365 -0.2416  0.3252 12.7804
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.900889   0.030549  94.958 <2e-16 ***

```

```
## typing_count 0.002342 0.001925 1.216 0.2240
## sessionIndex -0.219793 0.096391 -2.280 0.0226 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.003 on 8157 degrees of freedom
## Multiple R-squared: 0.05238, Adjusted R-squared: 0.05214
## F-statistic: 225.4 on 2 and 8157 DF, p-value: < 2.2e-16
cat("Summary for H model:\n")

## Summary for H model:
models$H_model_summary

## [[1]]
##
## Call:
## lm(formula = total_h_time ~ typing_count + sessionIndex, data = .x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.59295 -0.14320 -0.01161  0.12247  2.27354
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.9720704  0.0071814 135.359  <2e-16 ***
## typing_count  0.0005474  0.0004526   1.209   0.227
## sessionIndex -0.0181029  0.0226592  -0.799   0.424
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2358 on 8157 degrees of freedom
## Multiple R-squared: 0.008226, Adjusted R-squared: 0.007983
## F-statistic: 33.83 on 2 and 8157 DF, p-value: 2.336e-15
```

Task-3: Normality assesment

```
# Define the subjects of interest
selected_subjects <- c("s002", "s020", "s040", "s057")

# Ensure 'subject' exists before filtering
if (!"subject" %in% colnames(Total_attempts)) {
  stop("Error: 'subject' column not found in Total_attempts.")
}

# Filter model_data for selected subjects
model_data <- Total_attempts |>
  filter(subject %in% selected_subjects, rep >= 31) # Keep only relevant subjects

# Fit models individually for each subject
filtered_models <- model_data %>%
  group_by(subject) %>%
  nest() %>%
```

```

mutate(
  UD_model = map(data, ~ lm(total_ud_time ~ typing_count + sessionIndex, data = .x)), # Fit UD model
  UD_residuals = map(UD_model, residuals), # Extract UD residuals
  UD_fitted = map(UD_model, fitted.values), # Extract UD fitted values
  DD_model = map(data, ~ lm(total_dd_time ~ typing_count + sessionIndex, data = .x)), # Fit DD model
  DD_residuals = map(DD_model, residuals), # Extract DD residuals
  DD_fitted = map(DD_model, fitted.values), # Extract DD fitted values
  H_model = map(data, ~ lm(total_h_time ~ typing_count + sessionIndex, data = .x)), # Fit H model
  H_residuals = map(H_model, residuals), # Extract H residuals
  H_fitted = map(H_model, fitted.values), # Extract H fitted values
)

#QQ plot for UD model
# Load necessary libraries
library(dplyr)
library(tidyr)
library(purrr)

# Open a plotting window with 4 panels (2x2)
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

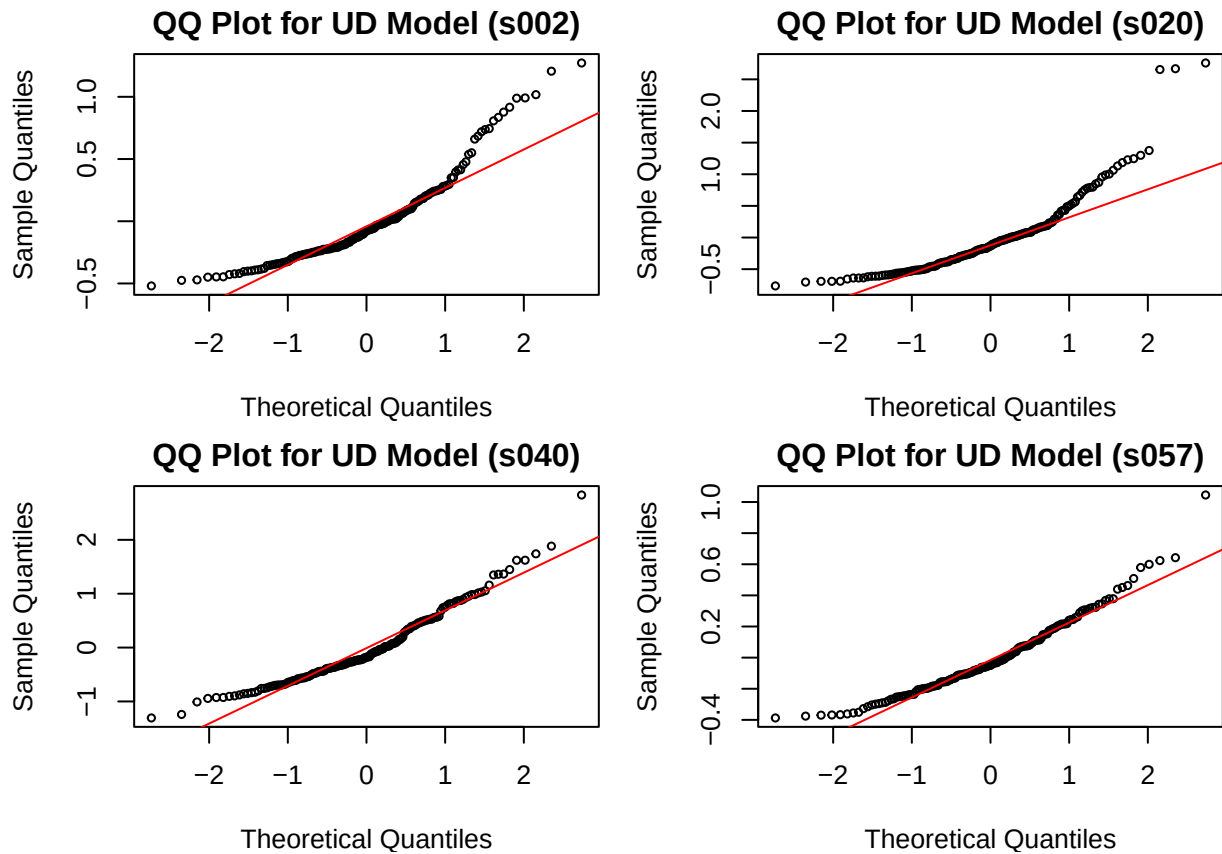
# Loop through each selected subject to create QQ plots
for (subject in selected_subjects) {

  # Extract residuals for the current subject
  residuals_UD <- unlist(filtered_models$UD_residuals[filtered_models$subject == subject])

  # Create QQ plot
  qqnorm(residuals_UD,
    main = paste("QQ Plot for UD Model (", subject, ")", sep = ""),
    xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",
    pch = 1, cex = 0.6) # Hollow circles with smaller size

  # Add a reference QQ line
  qqline(residuals_UD, col = "red")
}

```

```
# Reset plotting window
par(mfrow = c(1, 1))

#QQ plot for DD model

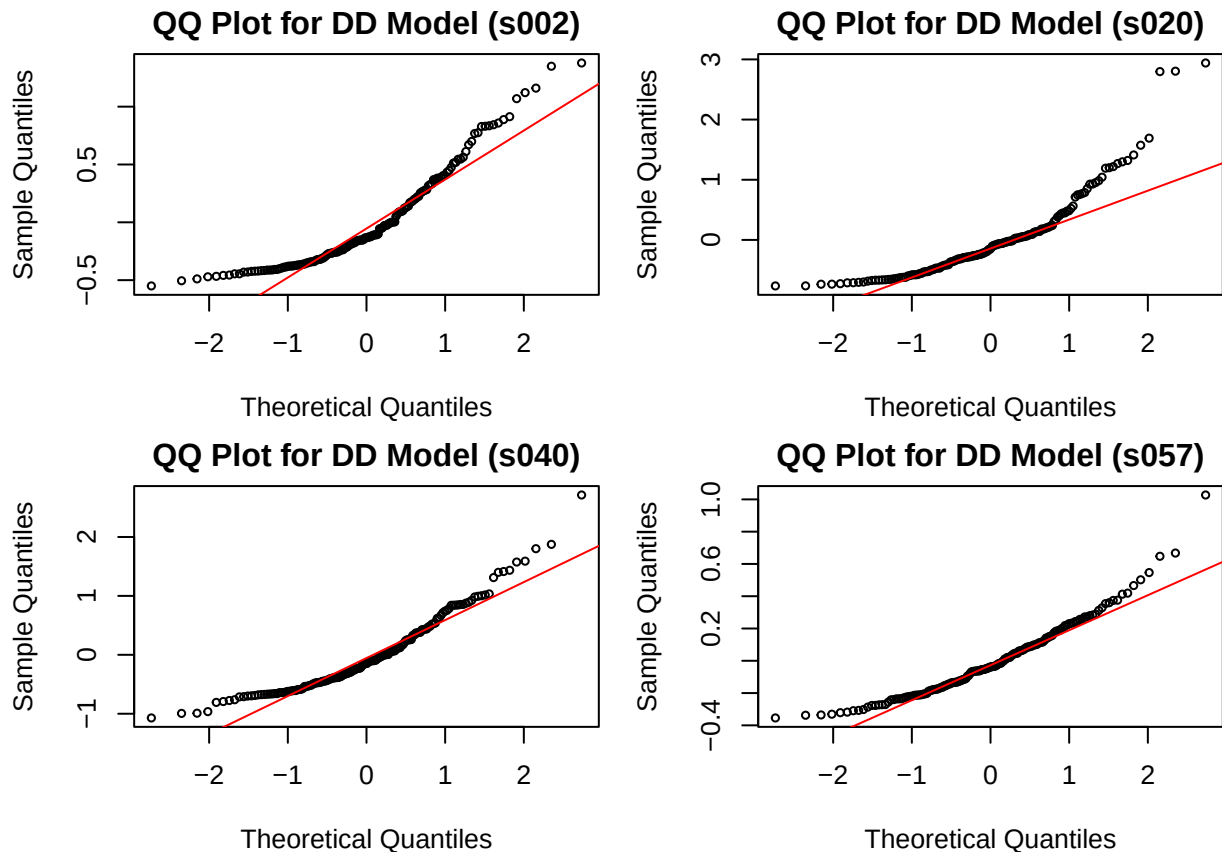
# Open a plotting window with 4 panels (2x2)
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

# Loop through each selected subject to create QQ plots
for (subject in selected_subjects) {

  # Extract residuals for the current subject
  residuals_DD <- unlist(filtered_models$DD_residuals[filtered_models$subject == subject])

  # Create QQ plot
  qqnorm(residuals_DD,
    main = paste("QQ Plot for DD Model (", subject, ")", sep = ""),
    xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",
    pch = 1, cex = 0.6) # Hollow circles with smaller size

  # Add a reference QQ line
  qqline(residuals_DD, col = "red")
}
```



```
# Reset plotting window
par(mfrow = c(1, 1))

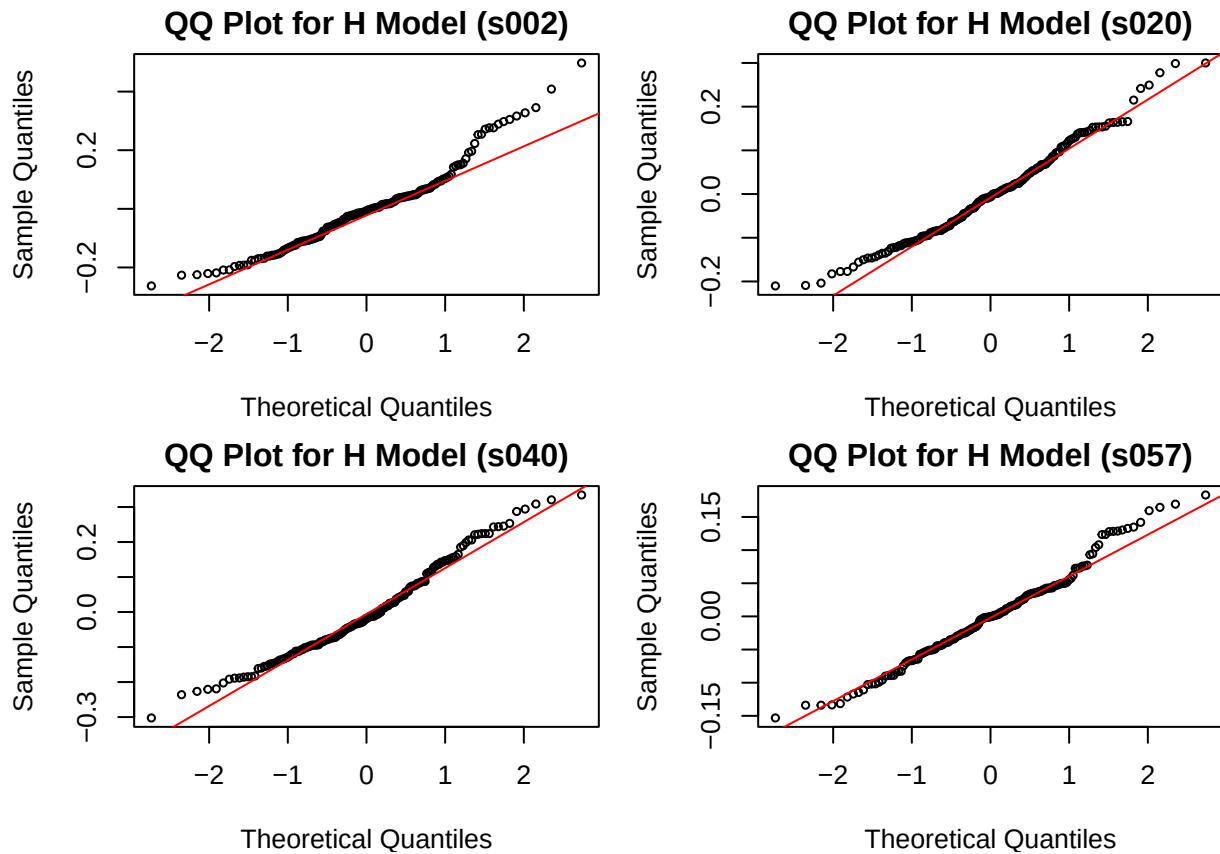
##QQ plot for H model
# Open a plotting window with 4 panels (2x2)
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

# Loop through each selected subject to create QQ plots
for (subject in selected_subjects) {

  # Extract residuals for the current subject
  residuals_H <- unlist(filtered_models$H_residuals[filtered_models$subject == subject])

  # Create QQ plot
  qqnorm(residuals_H,
    main = paste("QQ Plot for H Model (", subject, ")", sep = ""),
    xlab = "Theoretical Quantiles", ylab = "Sample Quantiles",
    pch = 1, cex = 0.6) # Hollow circles with smaller size

  # Add a reference QQ line
  qqline(residuals_H, col = "red")
}
```



```
# Reset plotting window
par(mfrow = c(1, 1))

#Residuals vs Fitted plot for UD

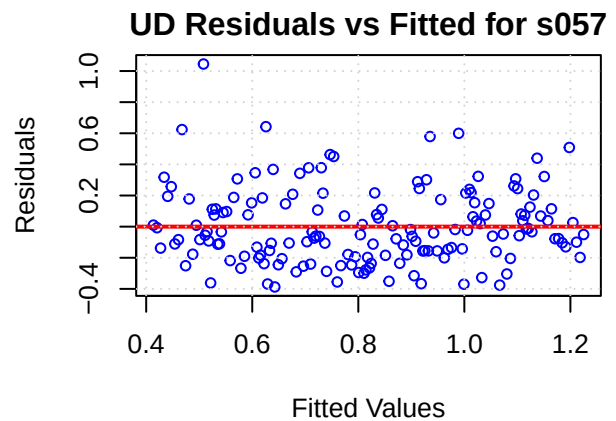
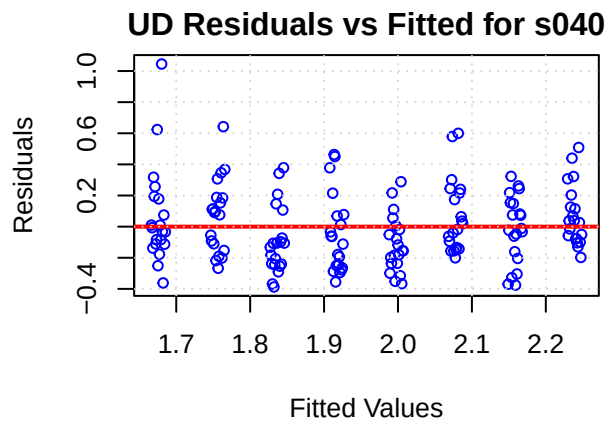
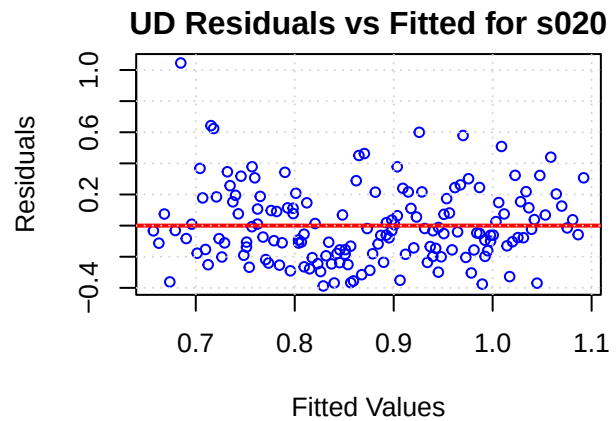
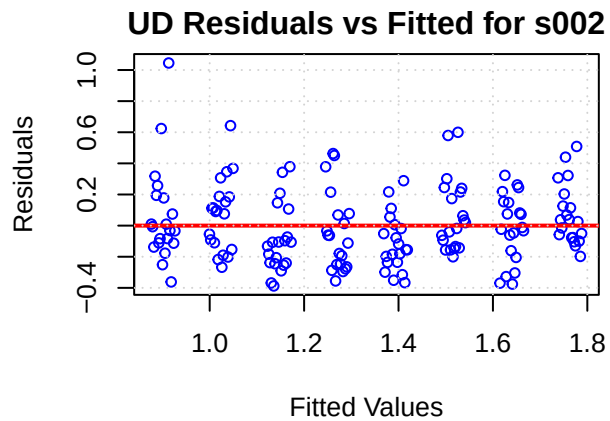
# Open a plotting window with 4 panels (2x2)
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

# Loop through each selected subject to create QQ plots
for (subject in selected_subjects) {

  # Extract residuals for the current subject
  fitted_values_UD <- unlist(filtered_models$UD_fitted[filtered_models$subject == subject])

  # Create the Residuals vs Fitted plot
  plot(fitted_values_UD, residuals_UD,
       main = paste("UD Residuals vs Fitted for", subject),
       xlab = "Fitted Values", ylab = "Residuals",
       col = "blue", pch = 1, cex = 0.8) # Hollow circles

  abline(h = 0, col = "red", lwd = 2) # Add a horizontal reference line
  grid() # Add gridlines
}
```



```
# Reset plotting window
par(mfrow = c(1, 1))

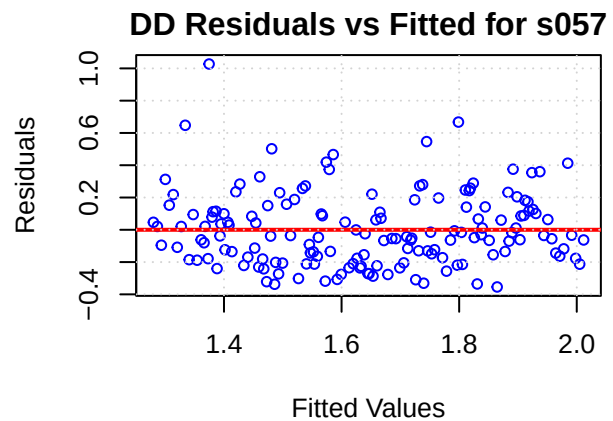
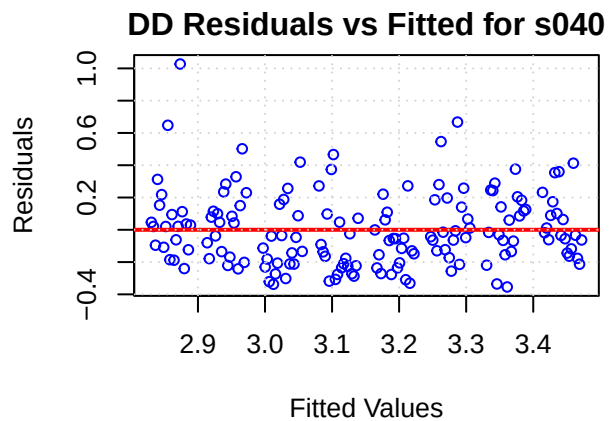
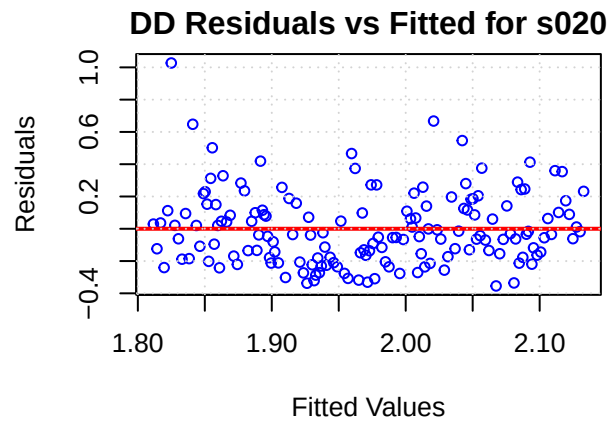
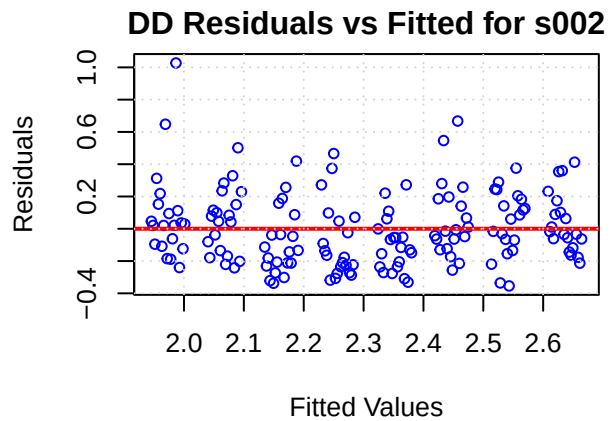
#Residuals vs Fitted plot for DD
# Open a plotting window with 4 panels
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

# Loop over the selected subjects to plot Residuals vs Fitted
for (subject in selected_subjects) {

  # Extract residuals and fitted values for the current subject
  fitted_values_DD <- unlist(filtered_models$DD_fitted[filtered_models$subject == subject])

  # Create the Residuals vs Fitted plot
  plot(fitted_values_DD, residuals_DD,
       main = paste("DD Residuals vs Fitted for", subject),
       xlab = "Fitted Values", ylab = "Residuals",
       col = "blue", pch = 1, cex = 0.8) # Hollow circles

  abline(h = 0, col = "red", lwd = 2) # Add a horizontal reference line
  grid() # Add gridlines
}
```



```
# Reset plotting window
par(mfrow = c(1, 1))

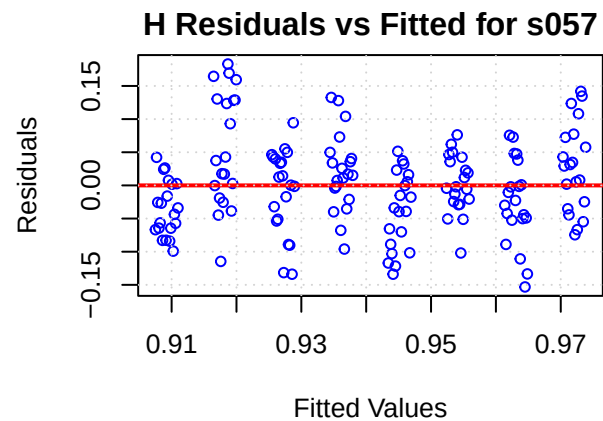
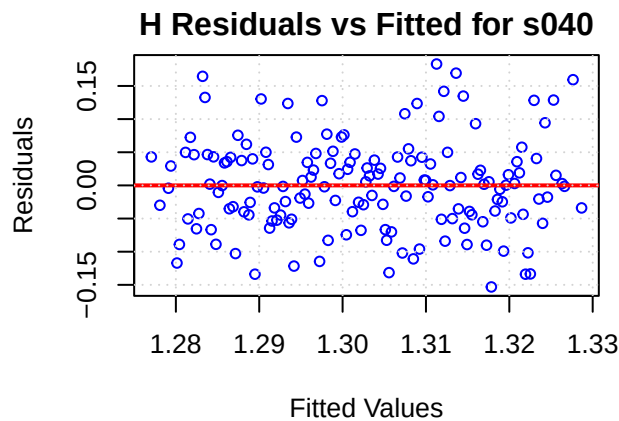
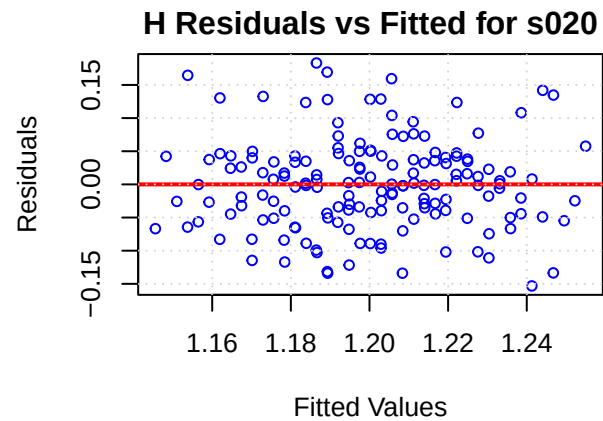
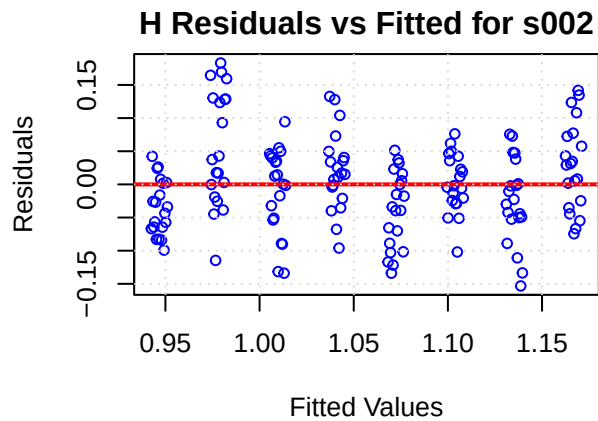
#Residuals vs Fitted plot for H
# Open a plotting window with 4 panels
par(mfrow = c(2, 2), bg = "white", mar = c(4, 4, 2, 1))

# Loop over the selected subjects to plot Residuals vs Fitted
for (subject in selected_subjects) {

  # Extract fitted values for the current subject
  fitted_values_H <- unlist(filtered_models$H_fitted[filtered_models$subject == subject])

  # Create the Residuals vs Fitted plot
  plot(fitted_values_H, residuals_H,
       main = paste("H Residuals vs Fitted for", subject),
       xlab = "Fitted Values", ylab = "Residuals",
       col = "blue", pch = 1, cex = 0.8) # Hollow circles

  abline(h = 0, col = "red", lwd = 2) # Add a horizontal reference line
  grid() # Add gridlines
}
```



```
# Reset plotting window
par(mfrow = c(1, 1))
```

```
#Shapiro-Wilk Test for Normality Distribution
```

```
# Perform Shapiro-Wilk test for UD, DD, and H models
```

```
# Function to perform Shapiro-Wilk test for residuals
```

```
perform_shapiro_test <- function(model_residuals, model_name, subject) {
  shapiro_test <- shapiro.test(model_residuals)
  return(data.frame(Subject = subject, Model = model_name, Statistic = shapiro_test$statistic, p_value = shapiro_test$p.value))
}
```

```
# UD Model - Shapiro-Wilk Test
```

```
shapiro_results_UD <- lapply(selected_subjects, function(subject) {
  residuals_UD <- unlist(filtered_models$UD_residuals[filtered_models$subject == subject])
  perform_shapiro_test(residuals_UD, "UD", subject)
})
```

```
# DD Model - Shapiro-Wilk Test
```

```
shapiro_results_DD <- lapply(selected_subjects, function(subject) {
  residuals_DD <- unlist(filtered_models$DD_residuals[filtered_models$subject == subject])
  perform_shapiro_test(residuals_DD, "DD", subject)
})
```

```
# H Model - Shapiro-Wilk Test
```

```
shapiro_results_H <- lapply(selected_subjects, function(subject) {
  residuals_H <- unlist(filtered_models$H_residuals[filtered_models$subject == subject])
  perform_shapiro_test(residuals_H, "H", subject)
})
```

```

perform_shapiro_test(residuals_H, "H", subject)
})

# Combine all Shapiro-Wilk results into one data frame
shapiro_results_all_df <- do.call(rbind, c(shapiro_results_UD, shapiro_results_DD, shapiro_results_H))

# View the Shapiro-Wilk test results for all models
print(shapiro_results_all_df)

##      Subject Model Statistic      p_value
## W      s002      UD 0.8889540 1.368663e-09
## W1     s020      UD 0.8328958 3.072920e-12
## W2     s040      UD 0.9447930 6.594598e-06
## W3     s057      UD 0.9473907 1.081771e-05
## W4     s002      DD 0.8860028 9.491160e-10
## W5     s020      DD 0.8333991 3.224413e-12
## W6     s040      DD 0.9220840 1.368816e-07
## W7     s057      DD 0.9383101 2.018171e-06
## W8     s002      H 0.9530234 3.307642e-05
## W9     s020      H 0.9800200 2.042552e-02
## W10    s040      H 0.9767892 8.516676e-03
## W11    s057      H 0.9856931 9.934050e-02

#False-Discovery Rate (FDR) - Benjamin-Hochberg

# Apply FDR correction to the p-values using p.adjust() with method = "fdr"
shapiro_results_all_df$fdr_adjusted_p_value <- p.adjust(shapiro_results_all_df$p_value, method = "fdr")

# View the updated results with FDR adjusted p-values
print(shapiro_results_all_df)

##      Subject Model Statistic      p_value fdr_adjusted_p_value
## W      s002      UD 0.8889540 1.368663e-09      4.105990e-09
## W1     s020      UD 0.8328958 3.072920e-12      1.934648e-11
## W2     s040      UD 0.9447930 6.594598e-06      1.130503e-05
## W3     s057      UD 0.9473907 1.081771e-05      1.622656e-05
## W4     s002      DD 0.8860028 9.491160e-10      3.796464e-09
## W5     s020      DD 0.8333991 3.224413e-12      1.934648e-11
## W6     s040      DD 0.9220840 1.368816e-07      3.285157e-07
## W7     s057      DD 0.9383101 2.018171e-06      4.036342e-06
## W8     s002      H 0.9530234 3.307642e-05      4.410189e-05
## W9     s020      H 0.9800200 2.042552e-02      2.228239e-02
## W10    s040      H 0.9767892 8.516676e-03      1.022001e-02
## W11    s057      H 0.9856931 9.934050e-02      9.934050e-02

# Apply log transformation to UD and DD times (adding 1 to avoid log(0))
Total_attempts1 <- Total_attempts %>%
  mutate(
    log_total_ud_time = log(total_ud_time + 1),
    log_total_dd_time = log(total_dd_time + 1)
  )

# Load necessary libraries
library(dplyr)
library(purrr)

```

```

# Re-run regression models using log-transformed UD and DD times
log_models <- Total_attempts1 %>%
  group_by(subject) %>%
  nest() %>%
  mutate(
    log_UD_model = map(data, ~ lm(log_total_ud_time ~ typing_count + sessionIndex, data = .x)),
    log_DD_model = map(data, ~ lm(log_total_dd_time ~ typing_count + sessionIndex, data = .x))
  )

# Function to perform Shapiro-Wilk test on residuals
perform_shapiro_test <- function(residuals, model_name, subject) {
  shapiro_test <- shapiro.test(residuals)
  return(data.frame(Subject = subject, Model = model_name, Statistic = shapiro_test$statistic, P_Value = shapiro_test$p.value))
}

# Initialize list to store results
log_shapiro_results_list <- list()

# Loop through selected subjects and perform normality tests
selected_subjects <- c("s002", "s020", "s040", "s057")

for (subject in selected_subjects) {

  # Extract index of the subject in log_models
  i <- which(log_models$subject == subject)

  # Extract residuals for log-transformed models
  residuals_log_UD <- residuals(log_models$log_UD_model[[i]])
  residuals_log_DD <- residuals(log_models$log_DD_model[[i]])

  # Perform Shapiro-Wilk test
  log_shapiro_results_list[[paste0(subject, "_log_UD")]] <- perform_shapiro_test(residuals_log_UD, "Log_UD", subject)
  log_shapiro_results_list[[paste0(subject, "_log_DD")]] <- perform_shapiro_test(residuals_log_DD, "Log_DD", subject)
}

# Combine results into a dataframe
log_shapiro_results_df <- do.call(rbind, log_shapiro_results_list)

# Apply FDR correction
log_shapiro_results_df$FDR_Adjusted_P_Value <- p.adjust(log_shapiro_results_df$P_Value, method = "fdr")

# View results
print(log_shapiro_results_df)

```

##	Subject	Model	Statistic	P_Value	FDR_Adjusted_P_Value
## s002_log_UD	s002	Log_UD	0.9490850	1.667249e-10	3.334498e-10
## s002_log_DD	s002	Log_DD	0.9384029	8.165310e-12	2.177416e-11
## s020_log_UD	s020	Log_UD	0.9528613	5.336656e-10	8.538650e-10
## s020_log_DD	s020	Log_DD	0.9334334	2.255468e-12	9.021874e-12
## s040_log_UD	s040	Log_UD	0.9865838	9.389335e-04	9.389335e-04
## s040_log_DD	s040	Log_DD	0.9677617	1.023861e-07	1.170127e-07
## s057_log_UD	s057	Log_UD	0.9591436	4.239033e-09	5.652044e-09
## s057_log_DD	s057	Log_DD	0.9249678	2.901569e-13	2.321255e-12

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this: